

Excel processing (Crealytics Package)

Last updated by | Gupta Vishal kailash | Jul 23, 2019 at 7:42 AM GMT+1

This Wiki provides an overview of how to use the Crealytics Scala package. This package allows reading and writing of Excel files with Apache Spark, via Spark SQL and DataFrames.

Once the Generic Ingestion pipeline has been created, an additional Wiki will be created to describe how to use this package in conjunction with the Anglian Water architecture, including how to configure the appropriate metadata in the BI System database.

Prerequisites

The library should have already been imported to the cluster via the Maven repository and requires Spark 2.0_

Maven coordinate for current latest release:

com.crealytics:spark-excel_2.11:0.11.0

Basic example

Using Databricks Scala cell. Providing an example of how to read an already uploaded Excel file into a DataFrame, whilst inferring the schema and specifying a cell range. Note. This is using the alternative .excel method which for convenience, provides an implicit that wraps the DataFrameReader returned by spark.read and provides a .excel method which accepts all possible options and provides default values.

```
%scala
import org.apache.spark.sql._
import com.crealytics.spark.excel._

val df = spark.read.excel(useHeader = true, inferSchema = true, dataAddress = "'Sheet1'!A1:C6").load("/FileStore/tables/Test1.xlsx")

display(df)
```

Create a DataFrame from an Excel file

Further examples using the standard .read method:

```
import org.apache.spark.sql._

val spark: SparkSession = ???
val df = spark.read
  .format("com.crealytics.spark.excel")
  .option("dataAddress", "'My Sheet'!B3:C35") // Optional, default: "A1"
  .option("useHeader", "true") // Required
  .option("treatEmptyValuesAsNulls", "false") // Optional, default: true
  .option("inferSchema", "false") // Optional, default: false
  .option("addColorColumns", "true") // Optional, default: false
  .option("timestampFormat", "MM-dd-yyyy HH:mm:ss") // Optional, default: yyyy-mm-dd hh:mm:ss[.fffffffffff]
  .option("maxRowsInMemory", 20) // Optional, default None. If set, uses a streaming reader which can help with big files
  .option("excerptSize", 10) // Optional, default: 10. If set and if schema inferred, number of rows to infer schema from
  .option("workbookPassword", "pass") // Optional, default None. Requires unlimited strength JCE for older JVMs
  .schema(myCustomSchema) // Optional, default: Either inferred schema, or all columns are Strings
  .load("Worktime.xlsx")
```

Example DataFrame from an excel file using custom schema

```
import org.apache.spark.sql._
import org.apache.spark.sql.types._

val peopleSchema = StructType(Array(
  StructField("Name", StringType, nullable = false),
  StructField("Age", DoubleType, nullable = false),
  StructField("Occupation", StringType, nullable = false),
  StructField("Date of birth", StringType, nullable = false)))

val spark: SparkSession = ???
val df = spark.read
  .format("com.crealytics.spark.excel")
  .option("sheetName", "Info")
  .option("useHeader", "true")
  .schema(peopleSchema)
  .load("People.xlsx")
```

Write a DataFrame to an Excel file

```
import org.apache.spark.sql._

val df: DataFrame = ???
df.write
  .format("com.crealytics.spark.excel")
  .option("dataAddress", "'My Sheet'!B3:C35")
  .option("useHeader", "true")
  .option("dateFormat", "yy-mmm-d") // Optional, default: yy-m-d h:mm
  .option("timestampFormat", "mm-dd-yyyy hh:mm:ss") // Optional, default: yyyy-mm-dd hh:mm:ss.000
  .mode("append") // Optional, default: overwrite.
  .save("Worktime2.xlsx")
```

As you can see in the examples above, the location of data to read or write can be specified with the `dataAddress` option. Currently the following address styles are supported:

Data Address notation

B3: Start cell of the data. Reading will return all rows below and all columns to the right. Writing will start here and use as many columns and rows as required.

B3:F35: Cell range of data. Reading will return only rows and columns in the specified range. Writing will start in the first cell (B3 in this example) and use only the specified columns and rows. If there are more rows or columns in the DataFrame to write, they will be truncated. Make sure this is what you want.

'My Sheet'!B3:F35: Same as above, but with a specific sheet.

MyTable[#All]: Table of data. Reading will return all rows and columns in this table. Writing will only write within the current range of the table. No growing of the table will be performed.

For further information on the library see github:

<https://github.com/crealytics/spark-excel> 